# Récapitulatif des ajouts et décisions

Complément au dossier technique du MVP (v1.1) · 23 octobre 2026


- 1. Règle générale de repli (proposition)
- 2. Chaîne de repli par fonction
- 3. Manques et solution prévue
- 4. Capacité estimée (mes calculs, à vérifier)
- 5. Réseaux sociaux : ce qui est confirmé
- 6. Changements à faire dans le dossier (v1.2)
- 7. Tests de l’étape 1 et décision selon le résultat
- Sources (lues du 20 au 23 octobre 2026)


**Objet.** Ce récapitulatif reprend tout ce qui a été ajouté ou établi depuis le dossier technique v1.1 (qui contient déjà la connexion Google). Il ajoute ce qui était demandé : une **chaîne de repli** pour chaque fonction, la même pour tous les profils, et une **solution pour chaque manque**, afin que chaque fonctionnalité donne toujours un résultat. Ces éléments sont à intégrer dans un dossier v1.2.

**Légende des statuts.** ✔ confirmé par une page officielle lue · ◐ source tierce ou concurrente, à reconfirmer · ? non confirmé (un test tranche) · **proposition** = choix de conception de ma part, pas un fait sourcé. Une solution de repli n’est pas une confirmation : elle dit quoi faire si le test échoue. Les quotas changent souvent : tout est à reconfirmer dans les consoles. Chiffres lus les 20 à 23 octobre 2026.

# 1. Règle générale de repli (proposition)

Chaque fonction a une **chaîne ordonnée** de fournisseurs, du meilleur au moins bon, qui se termine par un **repli ultime** qui ne peut pas échouer (base propre, liens manuels ou saisie par l’utilisateur). La chaîne est **la même pour tous les profils** (visiteur, inscrit, insider, administrateur) : le profil ne change que la limitation de débit du §6 du dossier et les droits d’accès, jamais le choix du fournisseur. Les quotas forment un **pool commun**.

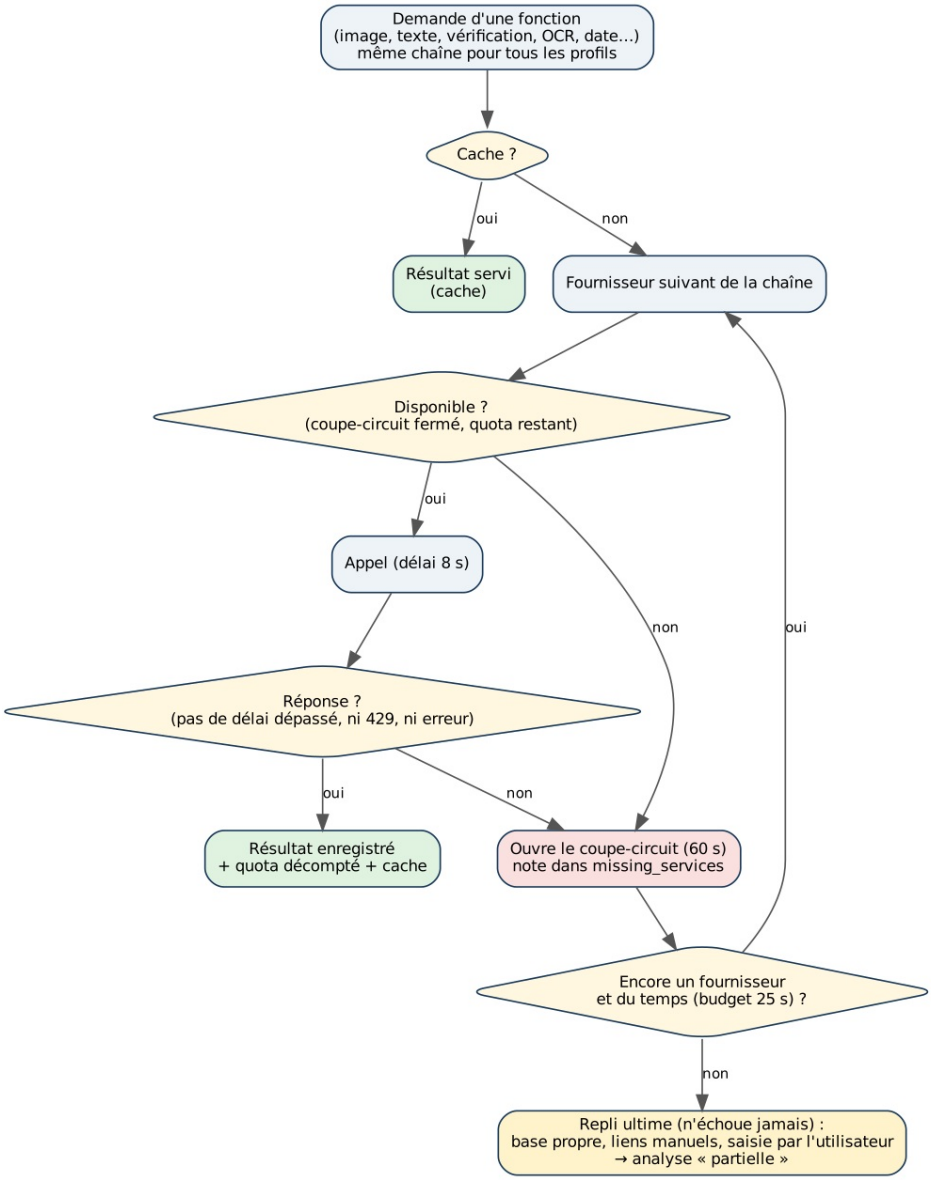



Figure 1 — Boucle de repli appliquée à chaque fonction

**Ce qui déclenche le passage au suivant** : délai de 8 s dépassé, erreur HTTP 429 ou 5xx, quota épuisé (`QuotaService.can_call()` faux), ou coupe-circuit ouvert. Un résultat vide n'est pas une panne : pas de repli, sauf l'option ci-dessous.

```
def run_chain(function, payload, budget=25):
    if (hit := cache.get(key(function, payload))): return hit
    missing = []
    for provider in CHAINS[function]:                 # même chaîne pour tous les profils
        if budget_exhausted(budget): break
        if breaker.is_open(provider.account) or not quota.can_call(provider.account):
            missing.append((provider.name, "indisponible ou quota")); continue
        try:
            result = provider.call(payload, timeout=min(8, time_left(budget)))
        except (Timeout, RateLimited, ProviderError) as e:
            breaker.open(provider.account, seconds=60); missing.append((provider.name, str(e))); continue
        quota.record(provider.account); cache.put(key, result); return result
    return manual_fallback(function, payload, missing)  # base propre + liens manuels -> résultat « partiel »
```

- **Échec rapide** (quota, 429, erreur) : suivant immédiat. **Délai dépassé** : suivant avec le temps restant du budget de 25 s. **Budget épuisé** : résultat partiel, ligne dans « Modules indisponibles », bouton « Relancer ».
- **Le coupe-circuit est par compte fournisseur, pas par moteur** : SerpApi propose plusieurs moteurs (Yandex, Lens) sous un même compte. Je ne sais pas si les moteurs partagent le quota (non confirmé), donc l'épuisement du compte ferme tous ses moteurs.
- **Option anti-faux-négatif** (proposition) : si le premier moteur d'image répond sans résultat et que son quota restant dépasse 50 %, interroger aussi le second.

- **Étalement du quota** (proposition) : plafond journalier par fournisseur = quota restant divisé par les jours restants du mois, pour qu'une journée chargée n'épuise pas le mois.
- **Journal** : chaque échec va dans `missing_services` et dans le tableau de bord (alerte à 80 %).

# 2. Chaîne de repli par fonction

| Fonction | Chaîne (dans l'ordre) | Repli ultime | Statut des éléments |
|---|---|---|---|
| **Image sur le web** (M4) | 1. SerpApi `yandex_images` → 2. SerpApi `google_lens` → 3. SearchAPI.io `google_lens` → 4. Apify (acteur Lens ou Yandex, tiers) | Empreinte pHash comparée à la base propre + liens manuels Lens, TinEye, Yandex | 1-2 : ✔ doc, quota 250/mois ✔, carte ? · 3 : ✔ doc, 100 requêtes une seule fois ◐, sans carte ✔ · 4 : ✔ crédit 5 $/mois sans carte, ◐ acteur tiers |
| **Texte sur le web** (M3) | 1. Tavily → 2. Serper → 3. SerpApi (moteur Google) → 4. SearchAPI.io (Google SERP) → 5. Exa | Recherche plein texte dans la base propre (wiki, fiches de sites, vérifications déjà vues) + liens manuels vers des moteurs | 1 : ✔ 1 000 crédits/mois sans carte · 2 : ◐ 2 500 une seule fois · 3-4 : ? moteurs de texte non vérifiés · 5 : ◐ crédits mensuels, article contradictoire |
| **Vérifications de faits** | 1. Fact Check Tools (`search` pour le texte, `imageSearch` pour l'image) | Table propre des vérifications déjà rencontrées + certifications des insiders | 1 : ✔ méthodes, ? gratuité et carte |
| **Texte dans une image** (OCR) | 1. OCR.space → 2. Tesseract.js dans le navigateur | Saisie ou collage du texte par l'utilisateur | 1 : ✔ 25 000/mois, 1 Mo, 500/jour/IP, français · 2 : ✔ existe, ? qualité |
| **Post de réseau social par lien** (M2) | Mastodon (API publique) ; Bluesky (API publique) ; TikTok (oEmbed) ; pour les autres : lecture refusée | Saisie du texte + envoi d'une capture ; le lien est conservé comme référence et le post est **affiché** par l'oEmbed officiel | Mastodon ✔ · Bluesky ✔ texte, ? image · TikTok ✔ description et vignette |
| **Date d'apparition** (M5) | 1. `DateResolver` sur 5 pages candidates → 2. plus ancienne capture de l'Internet Archive (CDX) | « Date inconnue », avec la mention de ce qui a été essayé | 1 : ✔ principe · 2 : ✔ champ `timestamp` et `oldest`, ? limites officielles |
| **IA et chatbot** (M9) | Gemini → Groq → stub | Stub : liens issus de la base, sans IA | Déjà au dossier ; ? éligibilité de Gemini |
| **E-mails** | Brevo | Notification dans l'application ; envois reportés au lendemain si le plafond de 300/jour est atteint | ✔ 300/jour (lu plus tôt) ; aucun second fournisseur vérifié |
| **WhatsApp** (optionnel) | Message libre dans la fenêtre de 24 h | E-mail, puis application | Déjà au dossier ; ? moyen de paiement |
| **Connexion** | Google | E-mail et mot de passe | ✔ flux ; ? domaine vérifié en production |

# 3. Manques et solution prévue

| # | Manque | Solution | Qui tranche |
|---|---|---|---|
| 1 | SerpApi exige peut-être une carte | Repli sur SearchAPI.io (sans carte, 100 requêtes) puis Apify (5 $ par mois sans carte) ; sinon repli ultime pHash + liens manuels. Le MVP fonctionne dans tous les cas | Test d'inscription |
| 2 | Quota d'image faible (250 par mois, soit environ 8 par jour : mon calcul) | Cache d'image de 24 h, allongé à plusieurs jours (proposition) ; plafond journalier ; analyse « relancée plus tard » quand le quota est vide ; chaîne de repli | Vous (volume visé) |
| 3 | Tavily : phrase exacte et dates inconnues | Deux requêtes par analyse (phrase, mots-clés) ; en cas de mauvais résultat, passer au moteur suivant ; dates par `DateResolver` | Test sur 10 phrases connues |
| 4 | Aucune API ne donne la date de première apparition | `DateResolver`, puis plus ancienne capture de l'Internet Archive ; toujours « date + origine ». La date retenue est « la plus ancienne trouvée », jamais « la première » | Test sur 10 pages |
| 5 | Fact Check Tools : gratuité et carte inconnues | Si l'activation exige la facturation, le retirer de la chaîne : le statut « contredite » vient alors de la table propre et des certifications des insiders | Test d'activation |
| 6 | Les moteurs d'image exigent une URL publique ; l'utilisateur envoie un fichier | Hébergement éphémère de l'image sur notre API : URL à jeton aléatoire, durée de quelques minutes, image gardée en mémoire (jamais en base), supprimée après l'analyse. La règle « ne pas conserver l'original » devient « conservé quelques minutes, jamais en base » (proposition). Sinon : SerpApi `image_id` (non vérifié) ou repli pHash | Vous (règle de conservation) ; test que les moteurs récupèrent l'URL |
| 7 | OCR non testé en français | Chaîne OCR.space → Tesseract.js → saisie manuelle ; réduire l'image sous 1 Mo avec Pillow avant l'envoi (proposition) | Test sur une capture réelle |
| 8 | Posts Meta et X non lisibles automatiquement | Saisie par l'utilisateur ; post affiché par l'oEmbed officiel ; extraction automatique seulement pour Mastodon, Bluesky, TikTok après test. Pour X : demander l'accord écrit ou un avis juridique avant toute extraction | Vous (décision de conception) |

| | | | |
|---|---|---|---|
| 9 | Image d’un post Meta ou X | L’utilisateur envoie la capture : elle passe dans les chaînes image (M4) et OCR | — |
| 10 | Conditions d’utilisation des fournisseurs non lues | Les lire à l’étape 1 ; un fournisseur interdit est retiré de sa chaîne | Vous |
| 11 | Usage commercial ou non (Vercel Hobby, offres gratuites) | Par défaut : non commercial. Sinon héberger le front ailleurs et relire les conditions de chaque fournisseur | Vous |
| 12 | Google en production : domaine vérifié non confirmé | Mode Testing pour la bêta (100 testeurs, autorisations de 7 jours) ; connexion par e-mail comme repli ; domaine propre plus tard | Test avec « auth-dev » |
| 13 | Gemini : pays éligible ? | Chaîne Groq → stub, déjà prévue | Test |
| 14 | Cadre légal des données | Avis d’une personne compétente avant l’ouverture ; politique de confidentialité citant Google, Brevo, Meta, OCR.space et les fournisseurs de recherche | Vous |
| 15 | Insiders et administrateurs : exiger Google ? | **Proposition par défaut** : exiger un compte lié à Google pour ces deux rôles (peu nombreux, donc compatibles avec la limite de 100 testeurs) | Vous |
| 16 | Cloud Vision exclu (carte) | Remplacé par les chaînes image et OCR ci-dessus | Fait |
| 17 | Render se met en veille | Bandeau « réveil du serveur » ; pages publiques servies par le front ; migration documentée | Déjà au dossier |

# 4. Capacité estimée (mes calculs, à vérifier)

| Ressource | Quota | Estimation |
|---|---|---|
| Image : SerpApi | 250 par mois | environ 8 recherches par jour |
| Image : SearchAPI.io | 100, une seule fois | réserve de démarrage et de comparaison |
| Image : Apify | 5 $ par mois (à partir de 5 $ les 1 000 images avec correspondances) | un peu moins de 1 000 images par mois avant les frais de démarrage |
| Texte : Tavily | 1 000 crédits par mois | environ 500 analyses par mois, à raison de deux requêtes par analyse |
| Texte : Serper | 2 500, une seule fois | environ 1 250 analyses |
| OCR : OCR.space | 25 000 par mois ; 500 par jour et par IP | ne limite pas le MVP |

Avec les cumuls, le MVP tient pour un pilote de quelques dizaines d’analyses par jour au plus. Ce n’est pas un service public à fort trafic.

# 5. Réseaux sociaux : ce qui est confirmé

| Réseau | Texte | Image | Statut |
|---|---|---|---|
| Mastodon | Oui (`content`, HTML) | Oui (`media_attachments`) | ✔ Sans jeton pour un statut public (miroir non officiel de la doc) ; une instance en liste blanche peut répondre 401 |
| Bluesky | Oui (`record.text`) | ? | ✔ Endpoints publics sans authentification d’après la doc ; ? `getPosts` et structure des images |
| TikTok | Oui (`title`) | Vignette seulement | ✔ oEmbed sans clé mentionnée ; ? limites d’usage |
| X | Via oEmbed | Non | ✔ oEmbed sans authentification ; ✔ conditions qui interdisent le scraping sans accord écrit (2023) |
| Instagram, Facebook, Threads | ? | ? | ✔ oEmbed limité à l’affichage, analyse interdite ; légende et miniature retirées depuis novembre 2025 |
| Reddit | ? | ? | ◐ OAuth obligatoire ; gratuit pour usage non commercial éligible |
| YouTube, Telegram, WhatsApp | ? | ? | Rien de vérifié |

# 6. Changements à faire dans le dossier (v1.2)

| Section du dossier | Changement |
|---|---|
| §1, §4, §7, §14.3, §17, §19 | Retirer Cloud Vision ; remplacer par les chaînes image et OCR |
| §2 (architecture) | Remplacer Vision par SerpApi, SearchAPI.io, Apify, Tavily, Serper, OCR.space ; ajouter le service d'hébergement éphémère d'image |
| §4 (comptes) | Ajouter SerpApi, SearchAPI.io, Tavily, Serper, OCR.space ; garder Google « auth » |
| §5 (modules) | `imagesearch` et `textsearch` : un `ProviderChain` commun ; `core` : coupe-circuit par fournisseur |
| §6 (moteur) | Ajouter la boucle de repli du §1 et l'étalement du quota |
| §8 | Réécrire « Recherche d'image » sans Vision ; ajouter l'hébergement éphémère |
| §14 (UML) | Ajouter `ProviderChain`, `CircuitBreaker`, `EphemeralImageStore` |
| §15 (IHM) | Ajouter « moteur utilisé », « Modules indisponibles », « Relancer », saisie assistée pour les posts de réseaux sociaux |
| §16, §17 | Tests de repli : simuler timeout, 429, quota épuisé, tous les fournisseurs en panne |
| §18 (plan) | Étape 1 : tests de la section 7 ci-dessous |
| §19 (risques) | Remplacer par la section 3 de ce récapitulatif |

# 7. Tests de l'étape 1 et décision selon le résultat

| Test | Si le test réussit | S'il échoue |
|---|---|---|
| Inscription SerpApi : carte demandée ? | Chaîne image complète | Commencer la chaîne image à SearchAPI.io |
| Requête `yandex_images` puis `google_lens` avec une image connue | Les deux sont gardés | Garder le moteur qui répond |
| Même image envoyée par URL éphémère de notre API | Hébergement éphémère adopté | Repli pHash seul pour les fichiers |
| Phrase connue entre guillemets sur Tavily, Serper | Chaîne texte confirmée | Le moteur suivant ; en dernier, base propre et liens manuels |
| Activation de Fact Check Tools sans facturation | Gardé en tête de sa chaîne | Retiré ; table propre et insiders |
| Capture de post en français sur OCR.space et Tesseract.js | Chaîne OCR confirmée | Saisie manuelle |
| Appel Mastodon, Bluesky et TikTok sur des posts publics | Lecture directe de ces trois réseaux | Saisie manuelle partout |
| Connexion Google en mode Testing | Bêta avec Google | Bêta avec e-mail seulement |
| Archive Internet : plus ancienne capture de 5 pages | Source de date de repli | « Date inconnue » |

# Sources (lues du 20 au 23 octobre 2026)

SerpApi : tarifs, Google Lens, Yandex inversé · SearchAPI.io : Google Lens, page produit, comparatif Scrapingdog · Apify : tarifs, Lens, Yandex · Yandex : tarifs, démarrage · Tavily : tarifs, crédits · Serper : fiche · Brave : article · Mojeek : page · Comparatif : Parallel · Fact Check Tools : doc, claims, imageSearch, Jentic · OCR : [OCR.space] (https://ocr.space/..), Tesseract.js, Mistral · Réseaux : X oEmbed, TechCrunch sur les conditions de X, Instagram oEmbed, Meta sans jeton, Iframely, Mastodon, Bluesky, TikTok, Reddit · Archive Internet : wayback, waybackpy · Google : OpenID Connect, audience · Cloud Vision : tarifs. Les pages changent : relire les versions en ligne avant de s'engager.